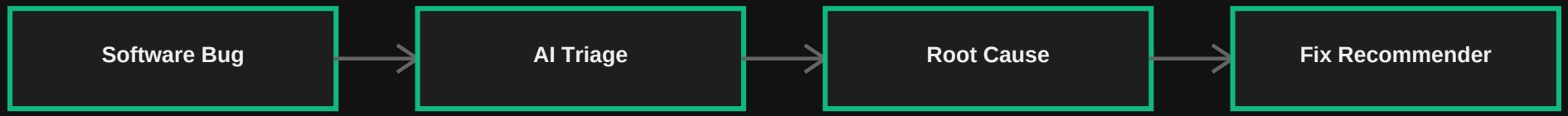
CREATION OF INTELLIGENT BUG DIAGNOSIS PLATFORM WITH FIX RECOMMENDATION ASSISTANCE GROUP

BugSense AI



Slide 2: Introduction

- Defect resolution represents a critical bottleneck in the software lifecycle, historically requiring intensive manual code tracing.
- Developers traditionally search historical reports, code databases, and reference manuals to isolate root cause anomalies.
- Runtime traces contain verbose logs where identifying the critical failure methods and calling chains is complex.
- BugSense AI automates diagnostic parsing and serves immediate fix recommendations by linking logs directly with RAG vector store playbooks.

Slide 3: Problem Statement

- Traditional debugging demands manual stack line inspections.
- Lack of knowledge reuse leads to fixing similar defects repeatedly.
- Sifting through massive log files creates cognitive overload.
- Diagnostic details remain siloed in developer communications.

TRADITIONAL DEBBUGGING

- Inspect raw logs manually row-by-row
- Manual web searches & API searches
- Guess root causes by code trials
- Fixes are undocumented & repeated later

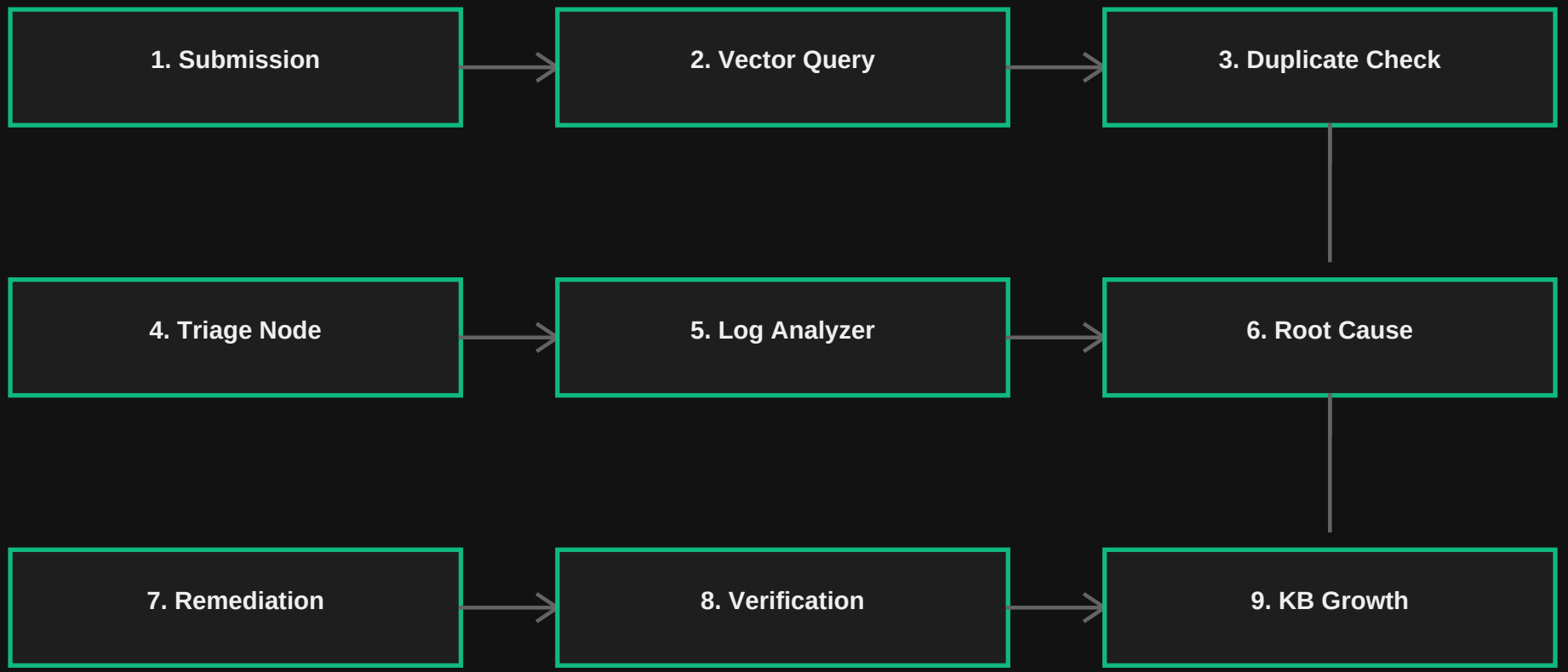
AI-ASSISTED DEBUGGING

- Automated agent parser extracts traces
- Vector similarity queries historical fixes
- Gemini LLM isolates probable root causes
- Closed-loop verify indexing database saves playbooks

Slide 4: Project Objectives

- Automate Defect Diagnostics: Analyze submitted bug tickets and runtime traces to classify categories and severity levels.
- Semantic Defect Retrieval: Query the FAISS vector database to retrieve historical bug reports using cosine similarity.
- Log Information Extraction: Parse stack traces to isolate exception classes, error logs, and timestamps.
- Contextual Remediation: Generate immediately actionable code recommendations and engineering checklists.
- Self-Growing Knowledge Base: Support resolution confirmation loops to continuously update vector database playbooks.

Slide 5: Proposed Solution Workflow



Slide 6: System Architecture Components

FRONTEND PORTAL

- React.js client interface
- Vite build configurations
- Tailwind CSS dashboard utilities
- Axios proxy connections

API GATEWAY SERVER

- Node.js + Express.js APIs
- Multer multi-part attachments
- Winston file logging wrapper
- Mongoose MongoDB database connections

AI SERVICE CORE

- FastAPI service wrapper
- LangGraph orchestrator machine
- FAISS similarity check algorithms
- Google Gemini reasoning engine

Slide 7: Technical Stack Details

Layer	Technology	Key Purpose
Frontend client	React, Vite, Tailwind CSS, Lucide Icons	Render diagnostic grids, charts & telemetry UI
API gateway	Node.js, Express.js, Multer, Winston	REST request proxy, log trace handling & routing
Database persistence	MongoDB (Mongoose), local JSON files	Saves bugs, verified resolutions & stats fallback
AI Microservice	Python 3.10, FastAPI	Wraps embeddings, similarity matches & agents
Vector database	FAISS, NumPy vector fallback	Maintains dense vectors for semantic similarity query
Agent runtime	LangChain, LangGraph StateGraph	Orchestrates transition state logic between agents
LLM Framework	Google Gemini API	Core inference engine for logs and recommendations
Document Parsing	PyPDF2, python-docx, pandas	Parses uploaded logs and structured bug files
Container topology	Docker, Docker Compose	Bundles multi-tier containers for fast setups

Slide 8: RAG & Semantic Retrieval Ingestion

- Vector Representation: Maps raw defect reports and log strings into a dense 384-dimensional space.
- FAISS Similarity Lookups: Calculates Euclidean distance metrics on embedding indexes to yield similar profiles.
- RAG Context Fusion: Enriches LLM generation by appending retrieved historical solutions to the prompt context.
- Keyword-Independent Retrieval: Bridges syntactic gaps, matching 'deadlock' logs against 'transaction exhaustion' vectors.

Slide 9: Multi-Agent LangGraph Node Transition

Triage Agent

Component category, severity rating & priority index

Log Analysis Agent

Extracts exceptions, call frame classes & methods

Duplicate Agent

FAISS similarity scans for prior ticket links

Root Cause Agent

Hypothesizes defects by comparative evidence mapping

Remediation Agent

Immediate code patches & best practice guides

Orchestration Control Node

State variables and log buffers are maintained across the LangGraph workspace state dictionary.

Slide 10: Knowledge Base Growth Loop

- Verification Gate: Submissions are never directly stored. Developers must confirm fixes via Resolution Verification.
- Threshold checking: A 90% cosine similarity threshold checks if the verified defect is already registered.

PATH A: Similarity > 90%

- The incoming defect matches an existing record.
- The system enriches the existing verified entry rather than creating duplicates.
- Overwrites matching FAISS index labels and updates metadata variables.

PATH B: Similarity <= 90%

- Represents a new software failure pattern.
- Creates a new unique vector entry and inserts the document into MongoDB.
- Regenerates FAISS index indices to add the new RAG playbook mapping.

Slide 11: Defect Pattern Analytics Dashboard

- Aggregation Routines: Group database documents by component category, priority queues, and timestamps.
- Category distribution: Reveals components with high defect frequency.
- Timelines & exception signatures: Captures recurrence trends and recurring log exceptions.
- Metric Exports: Compiles the structured analytics into CSV files for historical diagnostic audits.

Slide 12: End-to-End Ingestion Workflow

1. Submit log files & traces to React frontend portal.

2. Express router validates inputs and saves initial documents to DB.

3. AI service maps fields into sentence embeddings via SentenceTransformers.

4. FAISS index executes similarity search, returning similar bug records.

5. Triage Agent maps Component, Severity, and Priority rating tags.

6. Log Analysis Agent parses logs to extract exception stack details.

7. Root Cause and Remediation Agents output hypotheses and patches.

8. Dashboard inspect drawer renders complete diagnostics.

9. Developer fixes bug and confirms via Resolution Verification.

10. Index growth loop enriches existing files or indexes new vectors.

Slide 13: Testing & Evaluation Metrics

ACCURACY

100.0%

12/12 Defect Cases Passed

LATENCY

22.9 ms

Avg FAISS Response

CONFIDENCE

85.4%

Avg Agent certainty

Metric Tested	Calculated Result	Benchmark Standard
Precision	100.0%	>= 90.0%
Recall	100.0%	>= 90.0%
F1 Score	100.0%	>= 90.0%
Knowledge Growth Actions	12 Created, 0 Updated	Self-growth verified

Slide 14: Conclusion & Future Enhancements

Conclusion

- Automates defect categorizations & environmental triage ratings.
- Bridges syntax discrepancies using RAG-based context retrieval.
- Coordinates deep log trace extraction via specialized agents.
- Self-grow knowledge base updates enrich historical memory stores.
- Evaluated diagnostic results reached a 100% E2E test pass rate.

Future Enhancements

- IDE Plugins: Native connectors to submit stack frames directly inside VS Code and PyCharm.
- Real-Time log streams: Interfacing directly with Datadog/ELK agent pipelines.
- RBAC controls: Enforcing user role permissions on verified resolutions.
- Code-level repair: Extending immediate fix suggestions to auto-patch generation models.

THANK YOU

Questions & Discussions